

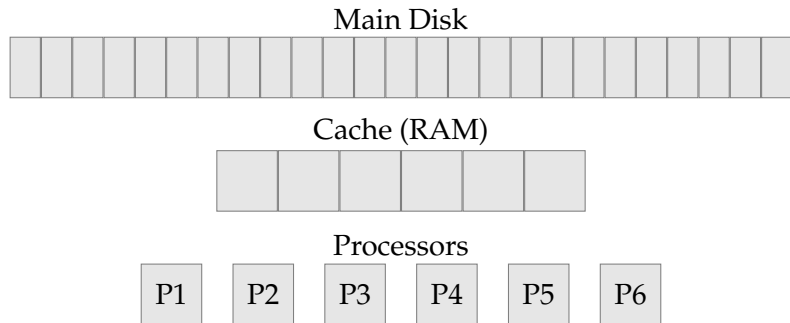
Increment-and-Freeze: LRU Miss Ratio Curves for Every Cache, Everywhere, All of the Time

Michael A. Bender, **Daniel DeLayo**, Bradley C. Kuszmaul,
William Kuszmaul, Evan West

Stony Brook University

2025

STANDARD PAGING MODEL: A MAIN DISK, A CACHE, AND PROCESSORS.



Cache Miss: accessing an element of the Main Disk costs 1.

Cache Hit: accessing an element in Cache costs 0.

The set of elements in cache is determined by *eviction policy*¹.

Cost: For sequence σ , the sum of the cost for each access.

¹ Least-Recently-Used eviction is widely used and performs well.

GOAL: HOW MUCH CACHE DO WE NEED?

When provisioning a physical or virtual system, someone has to decide how much RAM to give the system.

How much cache/RAM for a:

- ▶ video CDN?
- ▶ software download cache?
- ▶ database?
- ▶ graph algorithm?

During:

- ▶ the superbowl?
- ▶ 3am on a tuesday?
- ▶ a sale?
- ▶ release day?

The RAM/Cache requirements likely fluctuate!

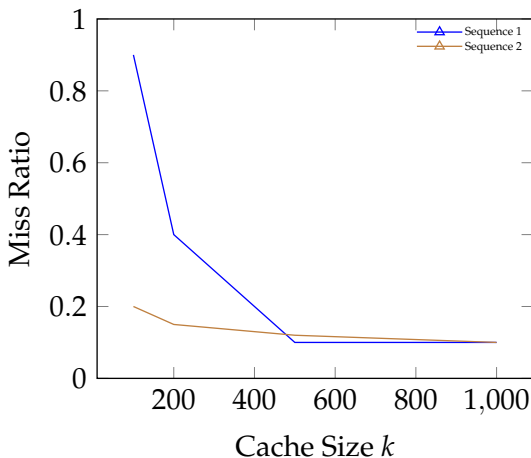
For LRU, this analysis is often done with Miss Ratio Curves.

LRU: MISS RATIO CURVES.

Let M_k be the cost of σ on a cache of size k running LRU.

Miss Ratio Curve: Mapping from k to $M_k/|\sigma|$.

Sample Miss Ratio Curve



STATE OF THE ART.

In 1975, Bennett and Kruskal solved miss ratio curve calculation with augmented trees.

PARDA [Niu et al., 2012] is a recent parallelization of this work that stores multiple copies of these trees.

Our algorithm Increment-and-Freeze (IaF) uses operations on ranges of elements, avoiding the downsides of parallel trees.

	PARDA	IaF	IaF Specializations ²
Work	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Parallelism	$O\left(\frac{M}{k}\right)$	$\Theta(\log n)$	$\Theta\left(\frac{n}{\log n}\right)$
Cache Misses	$\Theta(n \log n)$	$O\left(\frac{n}{B} \log n\right)$	$O\left(\frac{n}{B} \log_{\frac{M}{B}} \frac{n}{B}\right)$

M: Memory size. **B**: Block transfer size. **K**: max cache size

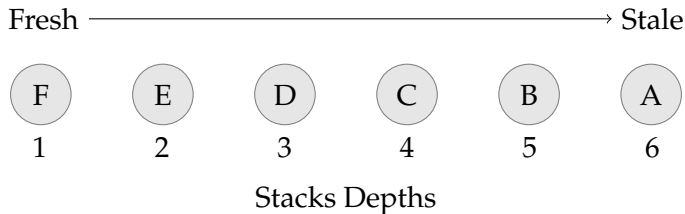
² We have versions that are either more parallel or more IO optimal.

THE CRUX OF MISS RATIO CURVES: STACK DEPTH.

Let C_k be a cache of size k running LRU on some sequence σ .

LRU is a Stack Algorithm. That is,

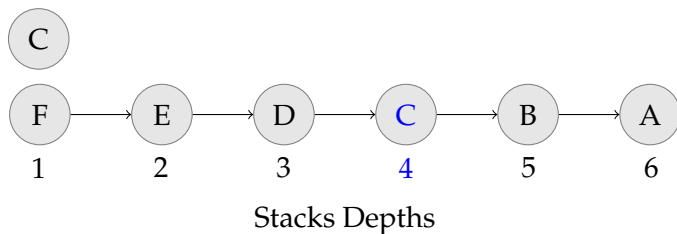
- ▶ For a request X , a hit on $C_k \implies$ a hit on C_{k+1} .
- ▶ **Stack Depth:** the smallest value of k on which X hits.
- ▶ An MRC is roughly a prefix sum of the stack depths!



1970: MOVE TO FRONT.

[Mattson et al., 1970] maintains a linked list of nodes.

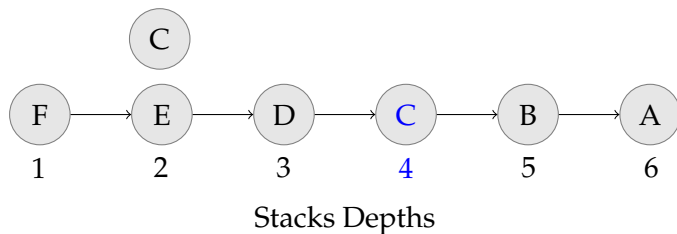
On access, the list is walked:



1970: MOVE TO FRONT.

[Mattson et al., 1970] maintains a linked list of nodes.

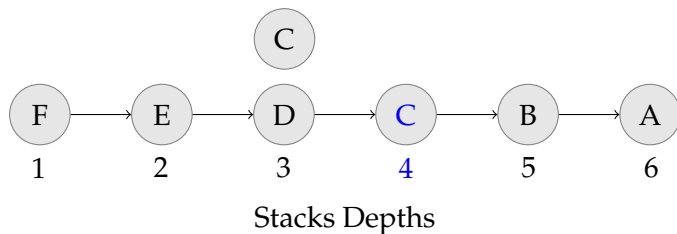
On access, the list is walked:



1970: MOVE TO FRONT.

[Mattson et al., 1970] maintains a linked list of nodes.

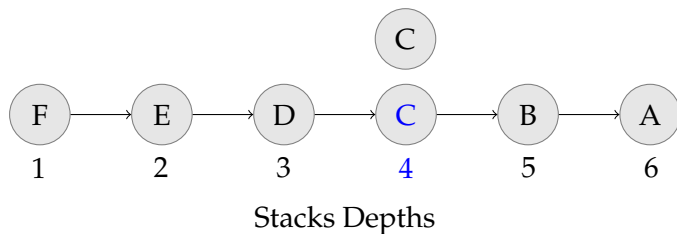
On access, the list is walked:



1970: MOVE TO FRONT.

[Mattson et al., 1970] maintains a linked list of nodes.

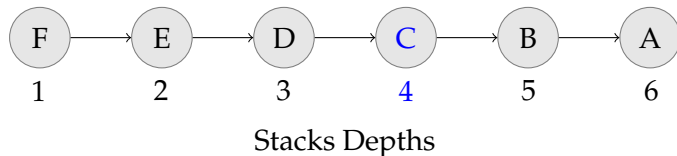
On access, the list is walked:



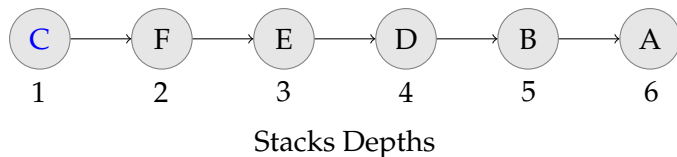
1970: MOVE TO FRONT.

[Mattson et al., 1970] maintains a linked list of nodes.

On access, the list is walked:



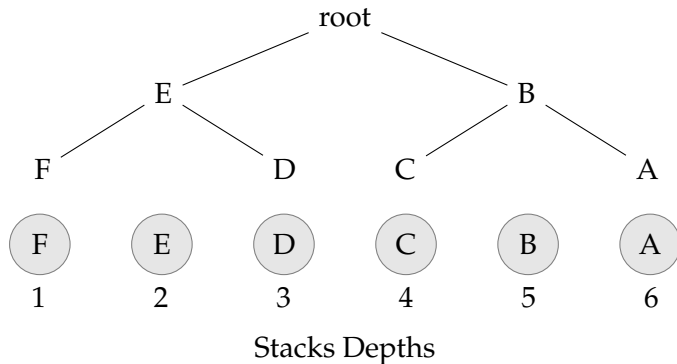
and it Moves To the Front.



1975: ORDER-STATISTICS TREES.

Stack Depth: Rank of an element sorted by access time.

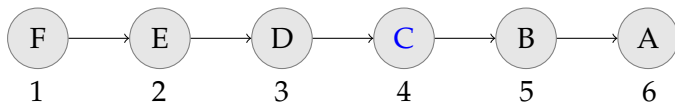
[Bennet and Kruskal, 1975] maintains an order-statistic tree.



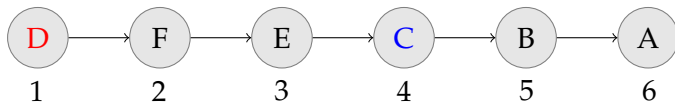
$O(n \log n)$ time, but poor cache locality and parallelism.

INTUITION: COUNT OF UNIQUE ACCESSSES.

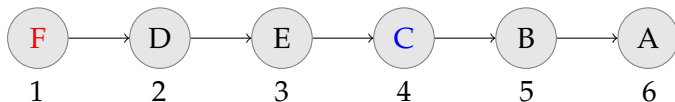
Stack Depth: Count of *unique* elements accessed between consecutive accesses to X.



Stacks Depths



Stacks Depths



Stacks Depths

STACK DEPTH: INCREMENTS AND FREEZES.

Let's incrementally maintain C 's stack depth.

Stack Depth: Count of *unique* elements accessed between consecutive accesses to X .

Consider the sequence:



1

STACK DEPTH: INCREMENTS AND FREEZES.

Let's incrementally maintain C 's stack depth.

Stack Depth: Count of *unique* elements accessed between consecutive accesses to X .

Consider the sequence:



2

STACK DEPTH: INCREMENTS AND FREEZES.

Let's incrementally maintain C 's stack depth.

Stack Depth: Count of *unique* elements accessed between consecutive accesses to X .

Consider the sequence:



2

STACK DEPTH: INCREMENTS AND FREEZES.

Let's incrementally maintain C 's stack depth.

Stack Depth: Count of *unique* elements accessed between consecutive accesses to X .

Consider the sequence:



3

STACK DEPTH: INCREMENTS AND FREEZES.

Let's incrementally maintain C 's stack depth.

Stack Depth: Count of *unique* elements accessed between consecutive accesses to X .

Consider the sequence:



3

STACK DEPTH: INCREMENTS AND FREEZES.

Let's incrementally maintain C 's stack depth.

Stack Depth: Count of *unique* elements accessed between consecutive accesses to X .

Consider the sequence:



3

Accessing C again *Freezes* the stack depth. It will never change.

OUR ALGORITHM: RANGE INCREMENTS.

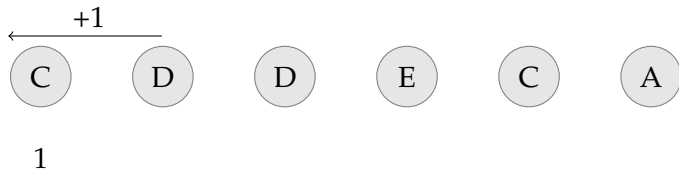
Increment-and-Freeze writes down a counter for each access.

Each access is a range update and freeze between occurrences.



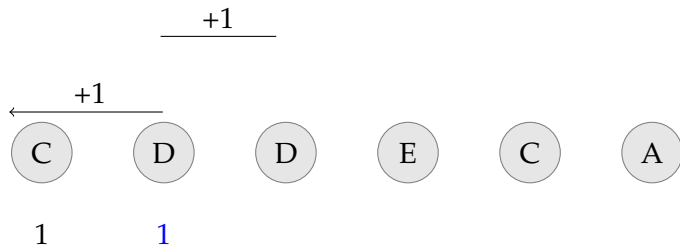
OUR ALGORITHM: RANGE INCREMENTS.

Increment-and-Freeze writes down a counter for each access.
Each access is a range update and freeze between occurrences.



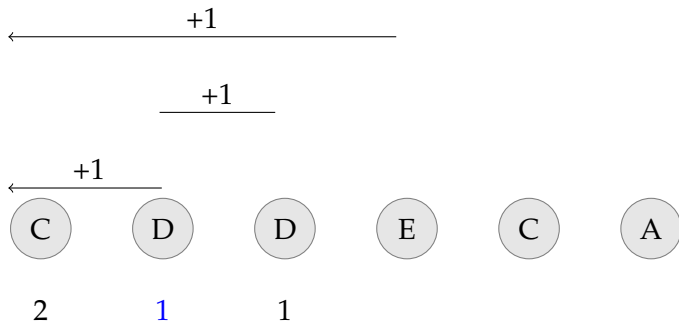
OUR ALGORITHM: RANGE INCREMENTS.

Increment-and-Freeze writes down a counter for each access.
Each access is a range update and freeze between occurrences.



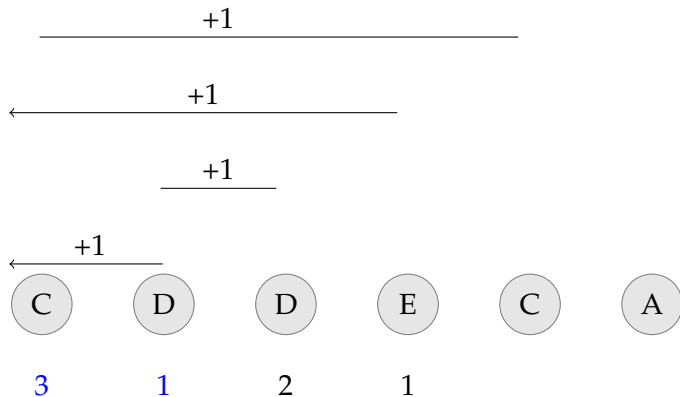
OUR ALGORITHM: RANGE INCREMENTS.

Increment-and-Freeze writes down a counter for each access.
Each access is a range update and freeze between occurrences.



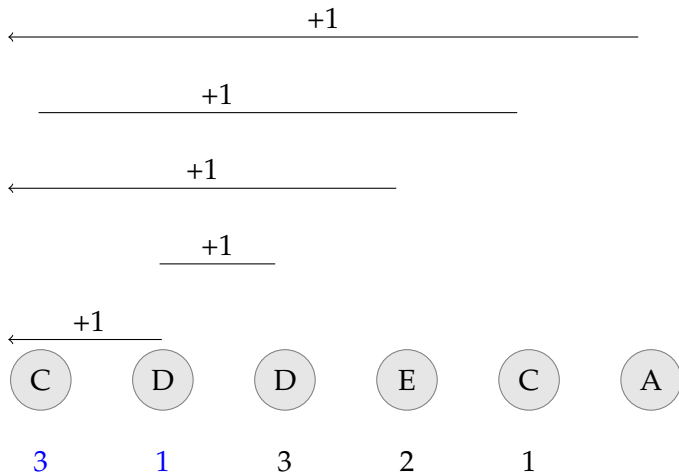
OUR ALGORITHM: RANGE INCREMENTS.

Increment-and-Freeze writes down a counter for each access.
Each access is a range update and freeze between occurrences.



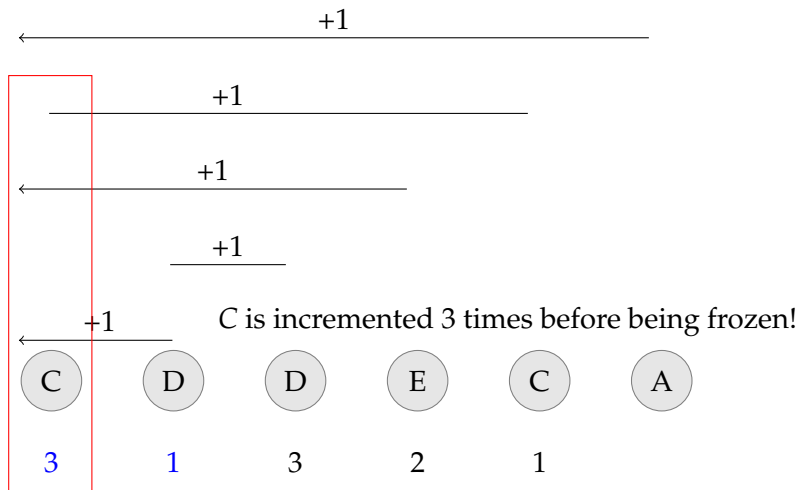
OUR ALGORITHM: RANGE INCREMENTS.

Increment-and-Freeze writes down a counter for each access.
Each access is a range update and freeze between occurrences.



OUR ALGORITHM: RANGE INCREMENTS.

Increment-and-Freeze writes down a counter for each access.
Each access is a range update and freeze between occurrences.

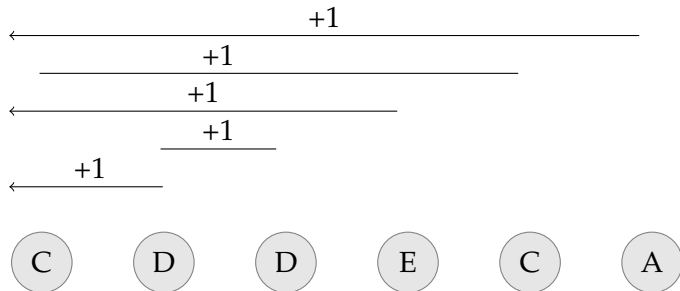


RANGE INCREMENT ANALYSIS.

Naïve computation is expensive: $O(n^2)$.

Instead, delay the updates and divide-and-conquer.

$O(1)$ -factor space overhead and a linear-time partition.

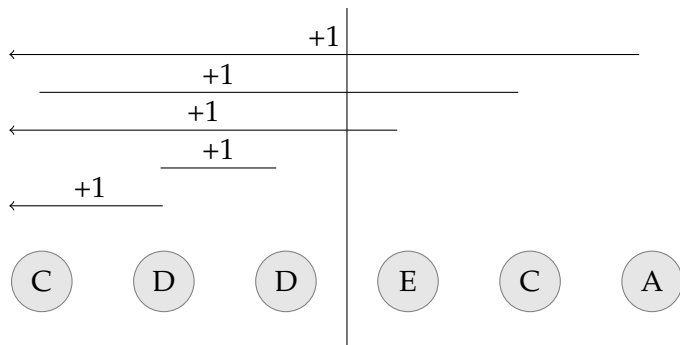


RANGE INCREMENT ANALYSIS.

Naïve computation is expensive: $O(n^2)$.

Instead, delay the updates and divide-and-conquer.

$O(1)$ -factor space overhead and a linear-time partition.

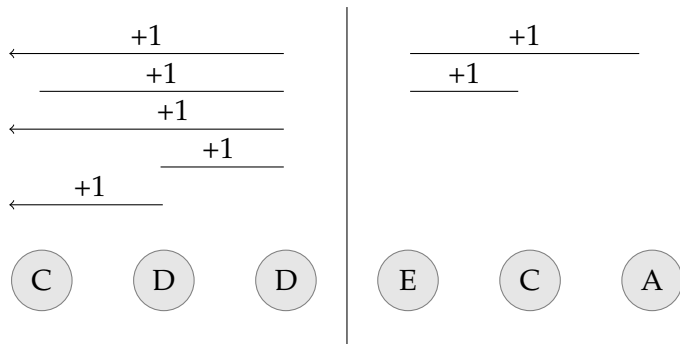


RANGE INCREMENT ANALYSIS.

Naïve computation is expensive: $O(n^2)$.

Instead, delay the updates and divide-and-conquer.

$O(1)$ -factor space overhead and a linear-time partition.

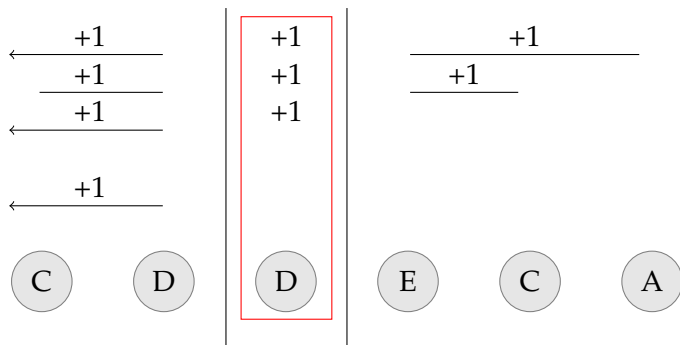


GETTING GOOD SPACE BOUNDS.

We store $O(1)$ information per line.

Two endpoints per line across all partitions.

Compress adjacent increments that span the entire partition.

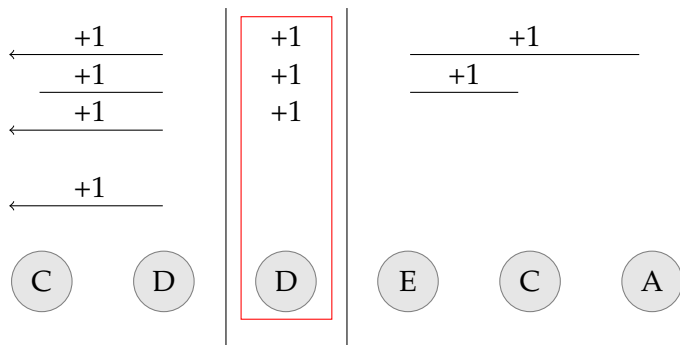


GETTING GOOD SPACE BOUNDS.

Great cache locality: we scan through the data.

Great parallelism: each partition is a disjoint problem.

Great space bounds: partitioning doesn't blow up the space.



INCREMENT-AND-FREEZE PERFORMS REALLY WELL.

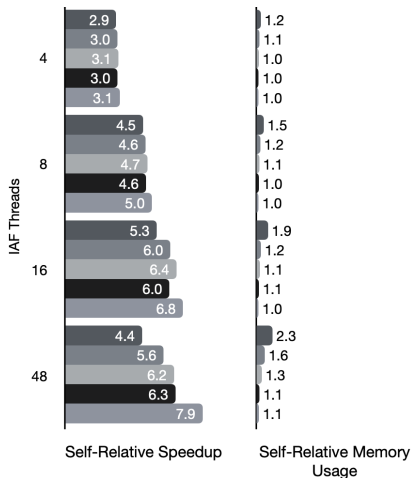
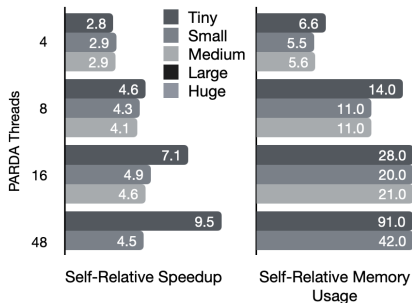
Average runtime in seconds.

System	Tiny	Small	Medium	Large	Huge
PARDA	32	198	1.3e+3	-	-
OS Tree	45	229	1.5e+3	3.6e+3	4.8e+4
IAF	11	32	176	377	-
Online-IAF	12	40	211	486	5.1e+3

Average memory usage in Megabytes.

System	Tiny	Small	Medium	Large	Huge
PARDA	31	623	2.9e+3	-	-
OS Tree	24	401	2.0e+3	1.5e+4	2.7e+4
IAF	957	2.7e+3	1.3e+4	2.9e+4	-
Online-IAF	35	573	2.6e+3	1.8e+4	3.5e+4

INCREMENT-AND-FREEZE PERFORMS REALLY WELL.



CONCLUSIONS.

Increment-and-Freeze uses a different algorithmic approach than prior work.

Designing Increment-and-Freeze for good cache performance and parallelism is great for runtime.

It's fast enough for *Every Cache, Everywhere, All of the Time*.

	PARDA	IaF	IaF Specializations ³
Work	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Parallelism	$O\left(\frac{M}{k}\right)$	$\Theta(\log n)$	$\Theta\left(\frac{n}{\log n}\right)$
Cache Misses	$\Theta(n \log n)$	$O\left(\frac{n}{B} \log n\right)$	$O\left(\frac{n}{B} \log_{\frac{M}{B}} \frac{n}{B}\right)$

M: Memory size. **B**: Block transfer size. **K**: max cache size

³ We have versions that are either more parallel or more IO optimal.